

Práctica: Persistencia del dominio con EF Core y migraciones

Paso a paso — Configurar DbContext, Fluent API y aplicar la primera migración sobre SQL Server

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 4 — Persistencia con Entity Framework Core
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Incorporar y verificar las entidades CentroCosto y Aprobacion en Domain, actualizando las referencias en Solicitud.
- Instalar y configurar Entity Framework Core 8 con el proveedor SQL Server en el proyecto Infrastructure.
- Crear la clase SolicitudesDbContext exponiendo los DbSet correspondientes a las entidades de dominio.
- Mapear entidades con Fluent API (EntityTypeConfiguration) manteniendo la pureza de la capa Domain.
- Configurar la cadena de conexión a SQL Server y habilitar la resiliencia con EnableRetryOnFailure.
- Generar la migración inicial mediante la CLI de .NET y aplicarla sobre la base de datos SQL Server.

2. Desarrollo paso a paso

Paso 1 — Paso previo: Incorporar CentroCosto y Aprobacion en el Dominio

Para que la base de datos contenga el modelo completo, aseguramos que existan las entidades CentroCosto y Aprobacion en Domain/Entidades.

Domain/Entidades/CentroCosto.cs y Aprobacion.cs

```
// Domain/Entidades/CentroCosto.cs
namespace SolicitudesApp.Domain.Entidades;

public class CentroCosto
{
    public int Id { get; set; }
    public stringCodigo { get; set; } = string.Empty;
    public string Nombre { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public ICollection<Solicitud> Solicitudes { get; set; } = new List<Solicitud>();
}

// Domain/Entidades/Aprobacion.cs
public class Aprobacion
{
    public int Id { get; set; }
    public int SolicitudId { get; set; }
    public Solicitud? Solicitud { get; set; }
    public int ResponsableId { get; set; }
    public Usuario? Responsable { get; set; }
    public EstadoSolicitud EstadoResultante { get; set; }
    public string? Comentario { get; set; }
    public DateTime Fecha { get; set; } = DateTime.UtcNow;
}
```

Modificaciones requeridas en Solicitud.cs: en Domain/Entidades/Solicitud.cs añade la clave foránea y navegación de Centro de Costo: `public int? CentroCostoId { get; set; }` y `public CentroCosto? CentroCosto { get; set; }`, además de la colección `public ICollection<Aprobacion> Aprobaciones { get; } = new List<Aprobacion>();` para registrar la trazabilidad.

Paso 2 — Instalar paquetes de EF Core para SQL Server

Instalamos el proveedor oficial de SQL Server en Infrastructure y las herramientas de diseño de EF Core en la API. En Windows PowerShell, desde la raíz de la solución SolicitudesApp:

PowerShell (Windows)

```
dotnet add SolicitudesApp.Infrastructure package Microsoft.EntityFrameworkCore.SqlServer
dotnet add SolicitudesApp.Infrastructure package Microsoft.EntityFrameworkCore.Design
dotnet add SolicitudesApp.Api package Microsoft.EntityFrameworkCore.Design
```

Paso 3 — Crear SolicitudesDbContext

En Infrastructure/Persistencia/SolicitudesDbContext.cs, declaramos la clase que administra la conexión y los conjuntos de entidades.

SolicitudesDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infrastructure.Persistencia;

public class SolicitudesDbContext : DbContext
{
    public SolicitudesDbContext(DbContextOptions<SolicitudesDbContext> options) : base(options) {

        public DbSet<Solicitud> Solicitudes => Set<Solicitud>();
        public DbSet<SolicitudDetalle> SolicitudDetalles => Set<SolicitudDetalle>();
        public DbSet<Usuario> Usuarios => Set<Usuario>();
        public DbSet<Rol> Roles => Set<Rol>();
        public DbSet<UsuarioRol> UsuarioRoles => Set<UsuarioRol>();
        public DbSet<Aprobacion> Aprobaciones => Set<Aprobacion>();
        public DbSet<MovimientoTesoreria> MovimientosTesoreria => Set<MovimientoTesoreria>();
        public DbSet<CentroCosto> CentrosCosto => Set<CentroCosto>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.ApplyConfigurationsFromAssembly(typeof(SolicitudesDbContext).Assembly);
        }
    }
}
```

ApplyConfigurationsFromAssembly: busca y registra automáticamente todas las clases que implementan `IEntityTypeConfiguration` en el proyecto, evitando tener que registrarlas una a una a mano.

Paso 4 — Configurar entidades con Fluent API

Creamos las clases de configuración en `Infrastructure/Persistencia/Configuraciones` para mapear `Solicitud`, `CentroCosto` y `Aprobacion` hacia SQL Server.

Configuraciones Fluent API

```
// SolicitudConfiguration.cs
builder.ToTable("Solicitudes");
builder.HasKey(s => s.Id);
builder.Property(s => s.Motivo).IsRequired().HasMaxLength(250);
builder.Property(s => s.MontoTotal).HasColumnType("decimal(18,2)");
builder.Property(s => s.Estado).HasConversion<int>();

builder.HasMany(s => s.Detalles).WithOne(d => d.Solicitud)
    .HasForeignKey(d => d.SolicitudId).OnDelete(DeleteBehavior.Cascade);

builder.HasOne(s => s.Solicitante).WithMany(u => u.Solicitudes)
    .HasForeignKey(s => s.SolicitanteId).OnDelete(DeleteBehavior.Restrict);

builder.HasOne(s => s.CentroCosto).WithMany(c => c.Solicitudes)
    .HasForeignKey(s => s.CentroCostoId).OnDelete(DeleteBehavior.Restrict);

// CentroCostoConfiguration.cs
builder.ToTable("CentrosCosto");
builder.HasKey(c => c.Id);
builder.Property(c => c.Codigo).IsRequired().HasMaxLength(20);
builder.Property(c => c.Nombre).IsRequired().HasMaxLength(100);
```

Paso 5 — Configurar la cadena de conexión de SQL Server y registrar el servicio

Configuramos la cadena de conexión a SQL Server en appsettings.json y registramos el DbContext con UseSqlServer en Program.cs.

appsettings.json y Program.cs

```
// appsettings.json:
// "ConnectionStrings": {
//   "DefaultConnection":
//     "Server=localhost;Database=SolicitudesDb;Trusted_Connection=True;TrustServerCertificate=True"
// }

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? "Server=(localdb)\\
mssqllocaldb;Database=SolicitudesDb;Trusted_Connection=True;TrustServerCertificate=True";

builder.Services.AddDbContext<SolicitudesDbContext>(options =>
{
    options.UseSqlServer(connectionString, sqlServerOptions =>
    {
        sqlServerOptions.EnableRetryOnFailure(
            maxRetryCount: 3,
            maxRetryDelay: TimeSpan.FromSeconds(5),
            errorNumbersToAdd: null);
    });
});
```

Parámetros clave de conexión a SQL Server: TrustServerCertificate=True permite conectar en desarrollo sin certificado SSL firmado. EnableRetryOnFailure activa la resiliencia automática ante caídas transitorias de red o reconexiones en SQL Server.

Paso 6 — Crear y aplicar la migración inicial en SQL Server

Generamos el script de migración en C# y aplicamos los cambios sobre la base de datos SQL Server. En Windows PowerShell, desde la raíz de la solución SolicitudesApp:

PowerShell (Windows)

```
dotnet ef migrations add MigracionInicial --project SolicitudesApp.Infrastructure --startup-
project SolicitudesApp.Api
dotnet ef database update --project SolicitudesApp.Infrastructure --startup-project
SolicitudesApp.Api
```

Creación automática en SQL Server: si la base de datos SolicitudesDb no existe en la instancia de SQL Server, EF Core la crea automáticamente junto a la tabla dbo.__EFMigrationsHistory para registrar los metadatos de migración.

3. Retos adicionales

1. Configura un índice único en la entidad Usuario para que no se puedan registrar dos usuarios con el mismo correo en SQL Server.
2. Implementa la configuración Fluent API para MovimientoTesoreria vinculando la relación 1 a 1 con Solicitud.

3. Implementa la configuración Fluent API para Aprobacion vinculando la clave foránea hacia Usuario (Responsable).
4. Inspecciona el script T-SQL que EF Core genera para SQL Server con el comando `dotnet ef migrations script`.
5. Prueba deshacer la migración aplicando `dotnet ef database update 0` y vuelve a aplicarla.

4. Entregables

- Las entidades `CentroCosto` y `Aprobacion` integradas en `Domain` con sus propiedades de navegación en `Solicitud`.
- La solución compila sin advertencias y la base de datos `SolicitudesDb` en SQL Server se genera con las tablas del dominio y `__EFMigrationsHistory`.
- Archivo de migración C# que describe las operaciones `Up()` y `Down()` del esquema relacional en T-SQL.
- Captura de pantalla o registro en terminal del comando `dotnet ef database update` completado con éxito.
- Comentario en código explicando por qué se configuró `UseSqlServer` con `EnableRetryOnFailure` y `DeleteBehavior.Restrict`.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 4 · Docente: Edwin Barrientos · New Horizons